

Advantages of CQRS

CQRS is well suited to complex domains and it provides a separation of concerns that helps to minimize and manage the complexity. Separating a system into query and command models makes it more maintainable, extensible, and flexible. Development teams can also be organized so that one team focuses on the query model while another focuses on the command model.

Segregating the responsibility between commands and queries can help to improve performance, scalability, and security. Performance can be improved by optimizing the schema specifically for each model. The schema for the query model can be optimized for queries, while the schema for the command model can be optimized for updates. Data in the query model's data store can be denormalized in order to increase the performance of queries that the application needs to execute.

Workloads between read and write operations will differ and using CQRS allows you to scale each of them independently of the others. Security is improved with CQRS because it makes it easier than with a single object model to ensure that only the right classes can update data.

Security can be easier to implement and test when using CQRS because each class is either used for reads or writes, but not both. This reduces the chance of inadvertently exposing data and operations that should not be available to a particular user in a certain context.

[Support](#) [Sign Out](#)